

# Final Project Part 2

## Data Parsing, Integration, Analysis, and Presentation

### Introduction:

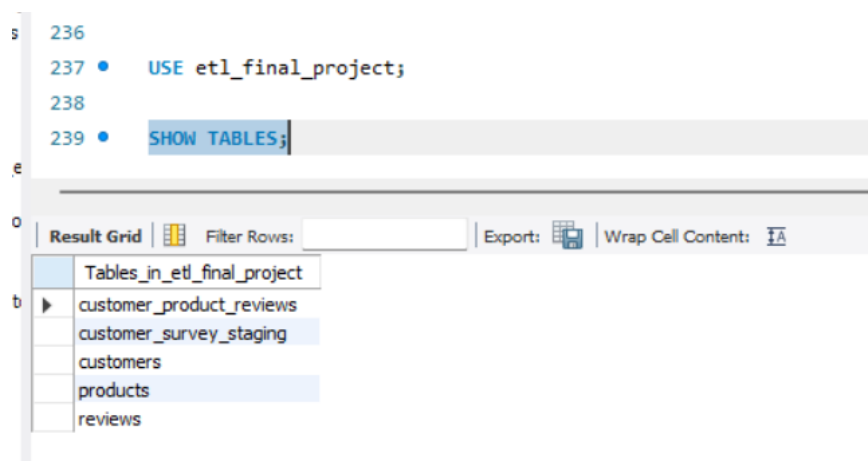
This project is a continuation of Final ETL Project – Part 1. The purpose of this project is to integrate, clean, validate, and analyze customer feedback data collected from multiple sources for an e-commerce company. The data comes from CSV survey files, JSON web feedback forms, and XML external review platforms.

In this project, Python and MySQL were used to parse and process the data, validate records, identify errors, and load clean data into relational database tables. After integrating the data, SQL queries were used to analyze customer ratings, identify common complaints, classify customer sentiment, and generate insights about product performance and customer satisfaction.

The project also includes database design, data validation, analysis reports, and presentation slides summarizing the key findings and recommendations for improving product quality and customer experience.

### Step 1: Connecting to the Existing Database

The existing database from Final Project Part 1 was successfully connected and verified before starting the data parsing and analysis process.



## Step 2: Creating Staging and Error Log Tables

Additional staging tables and an error logging table were created to support parsing and validation of JSON and XML data sources.

```
241
242 • USE etl_final_project;
243
244 -- Staging table for JSON web feedback
245 • CREATE TABLE IF NOT EXISTS web_feedback_staging (
246     feedback_id INT AUTO_INCREMENT PRIMARY KEY,
247     customer_id INT,
248     rating INT,
249     comments TEXT,
250     review_date DATE
251 );
252
253 -- Staging table for XML external reviews
254 • CREATE TABLE IF NOT EXISTS external_reviews_staging (
255     review_id INT AUTO_INCREMENT PRIMARY KEY,
256     customer_id INT,
257     rating INT,
258     comments TEXT,
259     review_date DATE
260 );
261
262 -- Error log table for parsing issues
263 • CREATE TABLE IF NOT EXISTS parsing_error_log (
264     log_id INT AUTO_INCREMENT PRIMARY KEY,
265     source_file VARCHAR(100),
266     row_identifier VARCHAR(100),
267     error_description TEXT,
268     log_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269 );
```

270

271 • SHOW TABLES;

| Result Grid                 |                          | Filter Rows: | Export: | Wrap Cell C |
|-----------------------------|--------------------------|--------------|---------|-------------|
| Tables_in_etl_final_project |                          |              |         |             |
| ▶                           | customer_product_reviews |              |         |             |
|                             | customer_survey_staging  |              |         |             |
|                             | customers                |              |         |             |
|                             | external_reviews_staging |              |         |             |
|                             | parsing_error_log        |              |         |             |
|                             | products                 |              |         |             |
|                             | reviews                  |              |         |             |
|                             | web_feedback_staging     |              |         |             |

### Step 3: Verifying the Default Product Record

A default product record with product\_id 999 was verified in the Products table. This record will be used for customer feedback entries that do not contain product information, helping maintain referential integrity between the Reviews and Products tables.

```
272
273 • USE etl_final_project;
274
275 • SELECT *
276 FROM products
277 WHERE product_id = 999;
```

Result Grid

Filter Rows:

Edit:

Export/Im

|   | product_id | name                      | category         | price |
|---|------------|---------------------------|------------------|-------|
| ▶ | 999        | General Customer Feedback | General Feedback | 0.00  |
| * | NULL       | NULL                      | NULL             | NULL  |

## Part 4: Data Parsing

### Step 4: Verifying the CSV Staging Table

The customer\_survey\_staging table was verified successfully to ensure that the CSV survey data was loaded correctly before starting the parsing and validation process.

```
280
281 • USE etl_final_project;
282
283 • SELECT *
284 FROM customer_survey_staging;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

|   | customer_id | name        | email                   | region | rating | comments         | review_date |
|---|-------------|-------------|-------------------------|--------|--------|------------------|-------------|
| ▶ | 1           | John Doe    | john.doe@example.com    | North  | 5      | Great product!   | 2024-01-01  |
|   | 2           | Jane Smith  | jane.smith@example.com  | West   | 4      | Fast delivery.   | 2024-01-02  |
|   | 3           | Emily Davis | emily.davis@example.com | South  | 3      | Average quality. | 2024-01-03  |

## Step 5: Checking for Missing Values in CSV Data

A validation query was executed to check for missing values in the CSV survey staging table. No missing values were found, indicating that the dataset was complete and ready for further validation and processing.

```
288 -- Check for missing values in CSV survey data
289
290 ● SELECT *
291 FROM customer_survey_staging
292 WHERE
293     name IS NULL
294     OR email IS NULL
295     OR region IS NULL
296     OR rating IS NULL
297     OR comments IS NULL
298     OR review_date IS NULL;
```

Result Grid | Filter Rows:  | Export: | Wrap Cell Content:

| customer_id | name | email | region | rating | comments | review_date |
|-------------|------|-------|--------|--------|----------|-------------|
|-------------|------|-------|--------|--------|----------|-------------|

## Step 6: Validating Rating Ranges

A validation query was executed to ensure that all rating values in the CSV survey data were within the acceptable range of 1 to 5. No invalid rating values were found.

```
301
302 -- Validate rating values in CSV survey data
303
304 ● SELECT *
305 FROM customer_survey_staging
306 WHERE rating < 1
307     OR rating > 5;
```

Result Grid | Filter Rows:  | Export: | Wrap Cell Content:

| customer_id | name | email | region | rating | comments | review_date |
|-------------|------|-------|--------|--------|----------|-------------|
|-------------|------|-------|--------|--------|----------|-------------|

## Step 7: Checking for Duplicate Records

A duplicate check was performed on the CSV survey data using GROUP BY and HAVING clauses. No duplicate records were found in the staging table.

```
309
310
311 -- Check for duplicate records in CSV survey data
312
313 SELECT
314     customer_id,
315     comments,
316     COUNT(*) AS duplicate_count
317 FROM customer_survey_staging
318 GROUP BY customer_id, comments
319 HAVING COUNT(*) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

| customer_id | comments | duplicate_count |
|-------------|----------|-----------------|
|-------------|----------|-----------------|

## Step 8: Loading Validated CSV Data into the Reviews Table

Validated CSV survey data was inserted into the Reviews table. During the validation process, duplicate records were identified and removed using a SQL DELETE query while preserving the original records. The final Reviews table contains clean, validated, and duplicate-free customer feedback data.

```
325 -- Insert validated CSV survey data into Reviews table
326
327 INSERT INTO reviews (
328     customer_id,
329     product_id,
330     rating,
331     comments,
332     review_date
333 )
334 SELECT
335     customer_id,
336     999,
337     rating,
338     comments,
339     review_date
340 FROM customer_survey_staging;
```

```
341
342 SELECT *
343 FROM reviews;
344
345
346 -- Temporarily disable safe update mode
347
348
349 SET SQL_SAFE_UPDATES = 0;
350
351 DELETE r1
352 FROM reviews r1
353 INNER JOIN reviews r2
354 ON r1.customer_id = r2.customer_id
355 AND r1.comments = r2.comments
356 AND r1.review_date = r2.review_date
357 AND r1.review_id > r2.review_id;
358
359 SET SQL_SAFE_UPDATES = 1;
```

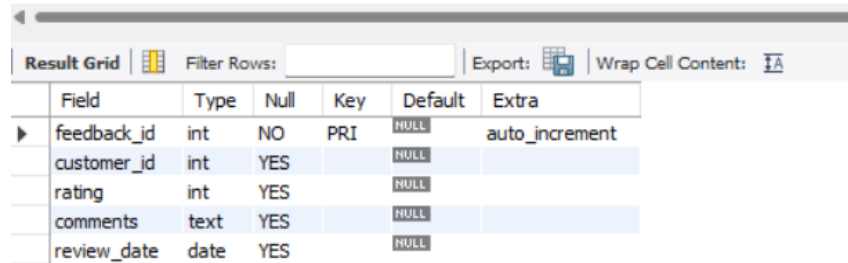
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

| review_id | customer_id | product_id | review_date | rating | comments         |
|-----------|-------------|------------|-------------|--------|------------------|
| 16        | 1           | 999        | 2024-01-01  | 5      | Great product!   |
| 17        | 2           | 999        | 2024-01-02  | 4      | Fast delivery.   |
| 18        | 3           | 999        | 2024-01-03  | 3      | Average quality. |
|           | NULL        | NULL       | NULL        | NULL   | NULL             |

## Step 9: Verifying the JSON Staging Table Structure

The structure of the `web_feedback_staging` table was verified to ensure that the JSON feedback data could be parsed and loaded correctly into the database.

```
363
364 -- Verify JSON staging table structure
365
366 • DESCRIBE web_feedback_staging;
```

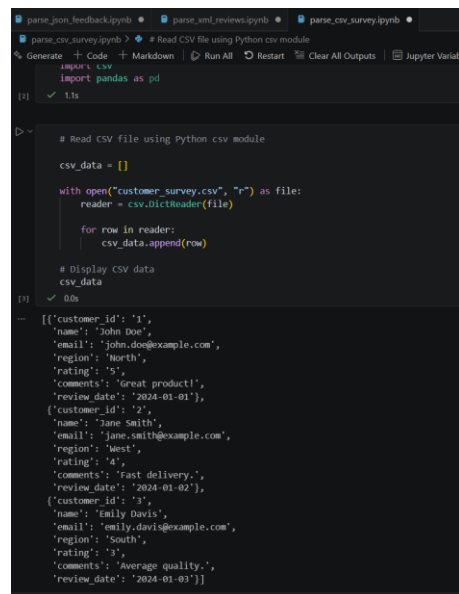


| Field       | Type | Null | Key | Default | Extra          |
|-------------|------|------|-----|---------|----------------|
| feedback_id | int  | NO   | PRI | NULL    | auto_increment |
| customer_id | int  | YES  |     | NULL    |                |
| rating      | int  | YES  |     | NULL    |                |
| comments    | text | YES  |     | NULL    |                |
| review_date | date | YES  |     | NULL    |                |

After verifying the JSON staging table structure in MySQL, Python was used to parse and process the JSON feedback data. The parsing script was developed to read the JSON file, validate the records, and prepare the data for insertion into the staging table.

## Additional CSV Parsing Using Python

A Python notebook named `parse_csv_survey.ipynb` was created to read and validate the CSV survey data using Python's built-in `csv` module. The CSV file was successfully loaded, and the survey records were extracted into Python data structures for further validation and analysis.



```
import csv
import pandas as pd

# Read CSV file using Python csv module
csv_data = []

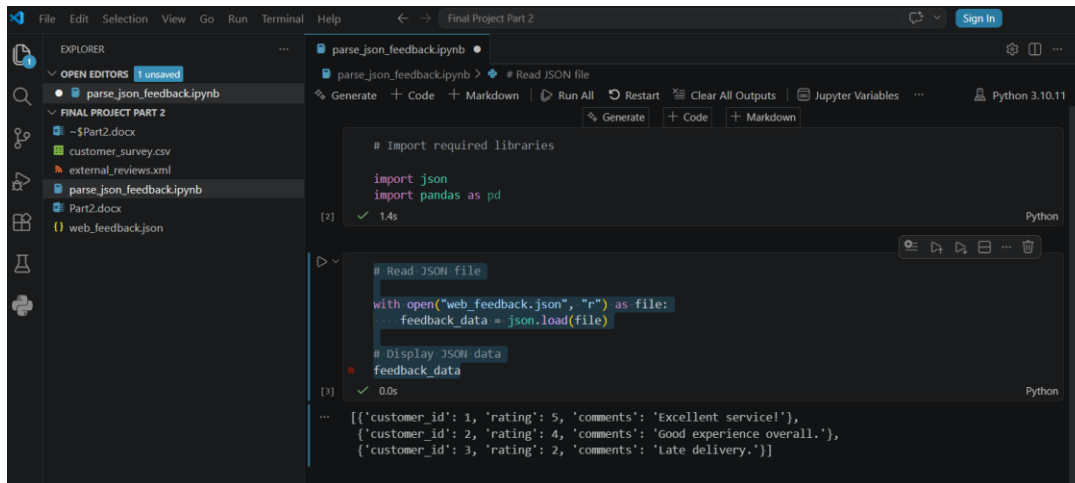
with open("customer_survey.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        csv_data.append(row)

# Display CSV data
csv_data
```

```
[{'customer_id': '1',
 'name': 'John Doe',
 'email': 'john.doe@example.com',
 'region': 'North',
 'rating': '5',
 'comments': 'Great product!',
 'review_date': '2024-01-01'},
 {'customer_id': '2',
 'name': 'Jane Smith',
 'email': 'jane.smith@example.com',
 'region': 'West',
 'rating': '4',
 'comments': 'Fast delivery.',
 'review_date': '2024-01-02'},
 {'customer_id': '3',
 'name': 'Emily Davis',
 'email': 'emily.davis@example.com',
 'region': 'South',
 'rating': '3',
 'comments': 'Average quality.',
 'review_date': '2024-01-03'}]
```

## Step 10: Reading the JSON Feedback File Using Python

Python libraries were imported and the JSON feedback file was successfully read using the json module. The contents of the file were displayed to verify that the feedback records were loaded correctly before starting the data transformation and validation process.



The screenshot shows a Jupyter Notebook interface with the following code and output:

```
# Import required libraries
import json
import pandas as pd

# Read JSON file
with open("web_feedback.json", "r") as file:
    feedback_data = json.load(file)

# Display JSON data
feedback_data
```

The output of the last cell is a list of three dictionaries:

```
[{'customer_id': 1, 'rating': 5, 'comments': 'Excellent service!'},
 {'customer_id': 2, 'rating': 4, 'comments': 'Good experience overall.'},
 {'customer_id': 3, 'rating': 2, 'comments': 'Late delivery.'}]
```

## Step 11: Converting JSON Data into a DataFrame

Python's pandas library was used to convert the JSON feedback data into a DataFrame structure. This transformation organized the records into tabular format, making the data easier to validate, clean, and process before loading it into the database.



The screenshot shows a Jupyter Notebook interface with the following code and output:

```
# Convert JSON data into a DataFrame
feedback_df = pd.DataFrame(feedback_data)

# Display DataFrame
feedback_df
```

The output of the last cell is a DataFrame with the following data:

|   | customer_id | rating | comments                 |
|---|-------------|--------|--------------------------|
| 0 | 1           | 5      | Excellent service!       |
| 1 | 2           | 4      | Good experience overall. |
| 2 | 3           | 2      | Late delivery.           |

## Step 12: Checking for Missing Values in JSON Data

A validation check was performed on the JSON feedback data to identify missing values in all columns. The results showed that no missing values were present in the dataset.

```
# Check for missing values in JSON data
feedback_df.isnull().sum()

5] ✓ 0.0s

customer_id    0
rating         0
comments      0
dtype: int64
```

## Step 13: Validating Rating Values in JSON Data

A validation check was performed on the JSON feedback data to identify invalid rating values outside the acceptable range of 1 to 5. No invalid ratings were found in the dataset.

```
# Validate rating values in JSON data

invalid_ratings = feedback_df[
    (feedback_df["rating"] < 1) |
    (feedback_df["rating"] > 5)
]

invalid_ratings

[6] ✓ 0.0s

... customer_id rating comments
```



## Step 14: Checking for Duplicate Records in JSON Data

A duplicate check was performed on the JSON feedback data using pandas. No duplicate records were found in the dataset.

```
# Check for duplicate records in JSON data

duplicate_records = feedback_df[
    ... feedback_df.duplicated()
]

duplicate_records
```

[7] ✓ 0.0s

| customer_id | rating | comments |
|-------------|--------|----------|
|-------------|--------|----------|

## Step 15: Adding a Review Date Column to JSON Data

A review\_date column was added to the JSON feedback DataFrame to match the structure of the MySQL staging table. This transformation prepared the JSON data for insertion into the database.

```
# Add review date column to JSON data

feedback_df["review_date"] = "2024-01-01"

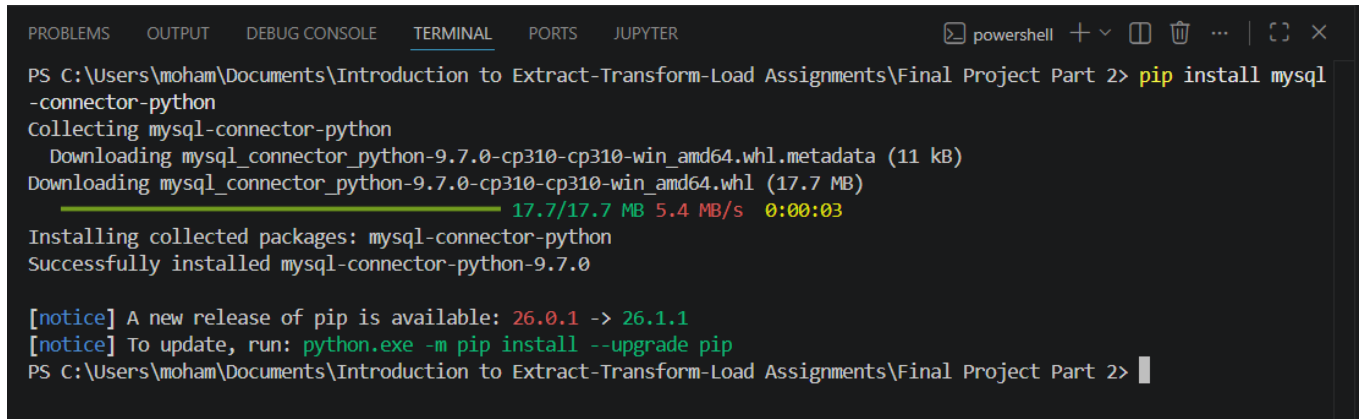
# Display updated DataFrame
feedback_df
```

[8] ✓ 0.0s

|   | customer_id | rating | comments                 | review_date |
|---|-------------|--------|--------------------------|-------------|
| 0 | 1           | 5      | Excellent service!       | 2024-01-01  |
| 1 | 2           | 4      | Good experience overall. | 2024-01-01  |
| 2 | 3           | 2      | Late delivery.           | 2024-01-01  |

## Step 16: Installing MySQL Connector for Python

The mysql-connector-python package was installed successfully to allow Python to connect to the MySQL database and insert parsed JSON data into the staging table.

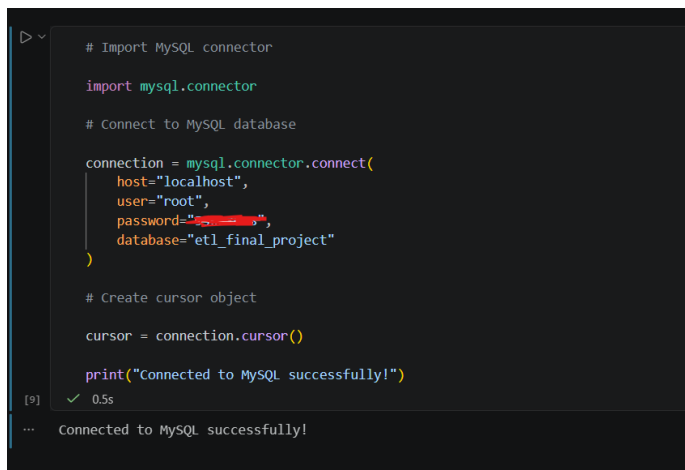


```
PS C:\Users\moham\Documents\Introduction to Extract-Transform-Load Assignments\Final Project Part 2> pip install mysql-connector-python
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.7.0-cp310-cp310-win_amd64.whl.metadata (11 kB)
  Downloading mysql_connector_python-9.7.0-cp310-cp310-win_amd64.whl (17.7 MB)
    17.7/17.7 MB 5.4 MB/s 0:00:03
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.7.0

[notice] A new release of pip is available: 26.0.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\moham\Documents\Introduction to Extract-Transform-Load Assignments\Final Project Part 2>
```

## Step 17: Connecting Python to the MySQL Database

Python was successfully connected to the MySQL database using the mysql-connector-python package. A database connection and cursor object were created to allow insertion of parsed JSON data into the staging table.



```
# Import MySQL connector
import mysql.connector

# Connect to MySQL database

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="etl_final_project"
)

# Create cursor object

cursor = connection.cursor()

print("Connected to MySQL successfully!")

[9] ✓ 0.5s
... Connected to MySQL successfully!
```

## Step 18: Inserting JSON Data into the MySQL Staging Table

Python was used to insert the validated JSON feedback data into the `web_feedback_staging` table in MySQL. A parameterized SQL INSERT query was executed for each record in the DataFrame using a loop and MySQL cursor object. The transaction was committed successfully, confirming that the parsed JSON data was loaded into the staging table without errors.

```
# Insert JSON data into web_feedback_staging table

insert_query = """
INSERT INTO web_feedback_staging
(customer_id, rating, comments, review_date)
VALUES (%s, %s, %s, %s)
"""

# Insert rows into table

for index, row in feedback_df.iterrows():
    cursor.execute(insert_query, (
        row["customer_id"],
        row["rating"],
        row["comments"],
        row["review_date"]
    ))

# Commit changes

connection.commit()

print("JSON data inserted successfully!")
```

[10] ✓ 0.0s

... JSON data inserted successfully!

## Step 19: Verifying Inserted JSON Data in MySQL

A verification query was executed in MySQL to confirm that the parsed JSON feedback records were successfully inserted into the `web_feedback_staging` table. The results showed that all records were loaded correctly into the database.

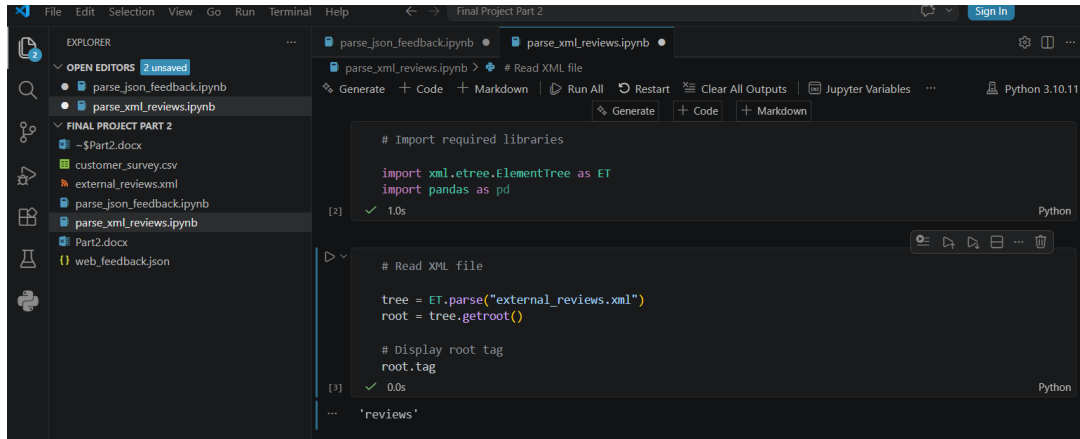
```
369
370 -- Verify inserted JSON records in staging table
371
372 • SELECT *
373 FROM web_feedback_staging;
```

Result Grid

|   | feedback_id | customer_id | rating | comments                 | review_date |
|---|-------------|-------------|--------|--------------------------|-------------|
| ▶ | 1           | 1           | 5      | Excellent service!       | 2024-01-01  |
|   | 2           | 2           | 4      | Good experience overall. | 2024-01-01  |
|   | 3           | 3           | 2      | Late delivery.           | 2024-01-01  |
| * | NULL        | NULL        | NULL   | NULL                     | NULL        |

## Step 20: Creating and Reading the XML Parsing Notebook

Python libraries required for XML parsing were imported successfully, and the `external_reviews.xml` file was read using the `xml.etree.ElementTree` library. The root element of the XML file was displayed to verify that the XML structure was loaded correctly before starting the data extraction and transformation process.



The screenshot shows a Jupyter Notebook with two cells. The first cell imports the `xml.etree.ElementTree` library as `ET` and `pandas` as `pd`. The second cell reads the `external_reviews.xml` file and displays the root tag, which is `'reviews'`.

```
# Import required libraries
import xml.etree.ElementTree as ET
import pandas as pd

# Read XML file
tree = ET.parse("external_reviews.xml")
root = tree.getroot()

# Display root tag
root.tag
```

Output: `'reviews'`

## Step 21: Extracting XML Review Records

The XML review records were extracted successfully using Python's `xml.etree.ElementTree` library. Customer IDs, ratings, and comments were retrieved from each review element and stored in a structured Python list for further processing and validation.



The screenshot shows a Jupyter Notebook cell with code that iterates through all `review` elements in the XML file. For each review, it extracts the `customer_id`, `rating`, and `comments` and appends them to a list called `xml_data`. The final output is a list of dictionaries containing the extracted data.

```
# Extract data from XML file

xml_data = []

for review in root.findall("review"):
    customer_id = review.find("customer_id").text
    rating = review.find("rating").text
    comments = review.find("comments").text

    xml_data.append({
        "customer_id": customer_id,
        "rating": rating,
        "comments": comments
    })

# Display extracted XML data

xml_data
```

Output:

```
[{'customer_id': '1', 'rating': '5', 'comments': 'Excellent service!'},
 {'customer_id': '2', 'rating': '4', 'comments': 'Good experience overall.'},
 {'customer_id': '3', 'rating': '2', 'comments': 'Late delivery.'}]
```

## Step 22: Converting XML Data into a DataFrame

The extracted XML review records were converted into a pandas DataFrame to organize the data into tabular format. This transformation simplified the validation, cleaning, and database loading processes.

```
# Convert XML data into a DataFrame

xml_df = pd.DataFrame(xml_data)

# Display DataFrame

xml_df
```

[5] ✓ 0.0s

|   | customer_id | rating | comments                 |
|---|-------------|--------|--------------------------|
| 0 | 1           | 5      | Excellent service!       |
| 1 | 2           | 4      | Good experience overall. |
| 2 | 3           | 2      | Late delivery.           |

## Step 23: Checking for Missing Values in XML Data

A validation check was performed on the XML review data to identify missing values in all columns. The results confirmed that no missing values were present in the dataset.

```
# Check for missing values in XML data

xml_df.isnull().sum()
```

[6] ✓ 0.0s

```
customer_id    0
rating         0
comments       0
dtype: int64
```

## Step 24: Validating Rating Values in XML Data

A validation check was performed on the XML review data to identify invalid rating values outside the acceptable range of 1 to 5. No invalid ratings were found in the dataset.

```
# Validate rating values in XML data

invalid_xml_ratings = xml_df[
    (xml_df["rating"].astype(int) < 1) |
    (xml_df["rating"].astype(int) > 5)
]

invalid_xml_ratings
```

[7] ✓ 0.0s

... customer\_id rating comments

## Step 25: Checking for Duplicate Records in XML Data

A duplicate check was performed on the XML review data using pandas. No duplicate records were found in the dataset.

```
# Check for duplicate records in XML data

duplicate_xml_records = xml_df[
    xml_df.duplicated()
]

duplicate_xml_records
```

[8] ✓ 0.0s

... customer\_id rating comments

## Step 26: Adding a Review Date Column to XML Data

A `review_date` column was added to the XML review DataFrame to match the structure of the MySQL staging table. This transformation prepared the XML data for insertion into the database.

```
# Add review date column to XML data

xml_df["review_date"] = "2024-01-01"

# Display updated DataFrame

xml_df
```

[9] ✓ 0.0s

|   | customer_id | rating | comments                 | review_date |
|---|-------------|--------|--------------------------|-------------|
| 0 | 1           | 5      | Excellent service!       | 2024-01-01  |
| 1 | 2           | 4      | Good experience overall. | 2024-01-01  |
| 2 | 3           | 2      | Late delivery.           | 2024-01-01  |

## Step 27: Connecting XML Parsing Notebook to the MySQL Database

Python was successfully connected to the MySQL database using the `mysql-connector-python` package. A database connection and cursor object were created to support insertion of parsed XML review data into the staging table.

```
# Import MySQL connector

import mysql.connector

# Connect to MySQL database

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="rootroot",
    database="etl_final_project"
)

# Create cursor object

cursor = connection.cursor()

print("Connected to MySQL successfully!")
```

[10] ✓ 0.2s

... Connected to MySQL successfully!

## Step 28: Loading XML Data into MySQL Staging Table

In this step, the cleaned XML review data was inserted into the `external_reviews_staging` table in MySQL using Python and `mysql-connector`. A loop was used to insert each row from the DataFrame into the database, and the transaction was committed successfully.

```
> <
# Insert XML data into external_reviews_staging table

insert_query = """
INSERT INTO external_reviews_staging
(customer_id, rating, comments, review_date)
VALUES (%s, %s, %s, %s)
"""

# Insert rows into table

for index, row in xml_df.iterrows():
    cursor.execute(insert_query, (
        row["customer_id"],
        row["rating"],
        row["comments"],
        row["review_date"]
    ))

# Commit changes

connection.commit()

print("XML data inserted successfully!")
[11] ✓ 0.0s
... XML data inserted successfully!
```

## Step 29: Verify XML Data in MySQL

In this step, a SQL `SELECT` query was executed to verify that the XML review data was successfully inserted into the `external_reviews_staging` table. The output confirmed that all records were stored correctly in MySQL, including customer IDs, ratings, comments, and review dates.

```
▶ <
# Verify inserted XML data

cursor.execute("SELECT * FROM external_reviews_staging")

rows = cursor.fetchall()

for row in rows:
    print(row)
[12] ✓ 0.0s
...
(1, 1, 5, 'Excellent service!', datetime.date(2024, 1, 1))
(2, 2, 4, 'Good experience overall.', datetime.date(2024, 1, 1))
(3, 3, 2, 'Late delivery.', datetime.date(2024, 1, 1))
```



# Step 30: Insert XML Data into Final Reviews Table

Validated XML review records were inserted from the external\_reviews\_staging table into the final Reviews table. A WHERE NOT EXISTS condition was used to prevent duplicate records and maintain data integrity. The final verification query confirmed that the XML review data was successfully integrated into the main reviews dataset.

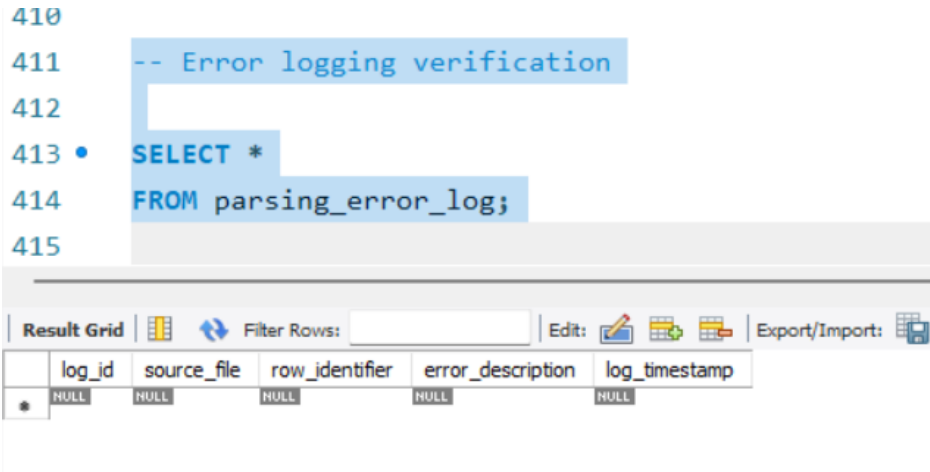
```
379      -- Insert XML staging data into final Reviews table without duplicates
380
381      INSERT INTO reviews (
382          customer_id,
383          product_id,
384          rating,
385          comments,
386          review_date
387      )
388      SELECT
389          customer_id,
390          999 AS product_id,
391          rating,
392          comments,
393          review_date
394      FROM external_reviews_staging
395      WHERE NOT EXISTS (
396          SELECT 1
397          FROM reviews r
398          WHERE r.customer_id = external_reviews_staging.customer_id
399              AND r.product_id = 999
400              AND r.rating = external_reviews_staging.rating
401              AND r.comments = external_reviews_staging.comments
402              AND r.review_date = external_reviews_staging.review_date
403      );
404
405      -- Verify final Reviews table after XML insertion
406
407      SELECT *
408      FROM reviews;
```

| Result Grid          |             |            |             |        |                          |  |
|----------------------|-------------|------------|-------------|--------|--------------------------|--|
| Filter Rows:         |             |            |             |        |                          |  |
| Edit: Export/Import: |             |            |             |        |                          |  |
| review_id            | customer_id | product_id | review_date | rating | comments                 |  |
| 16                   | 1           | 999        | 2024-01-01  | 5      | Great product!           |  |
| 17                   | 2           | 999        | 2024-01-02  | 4      | Fast delivery.           |  |
| 18                   | 3           | 999        | 2024-01-03  | 3      | Average quality.         |  |
| 22                   | 1           | 999        | 2024-01-01  | 5      | Excellent service!       |  |
| 23                   | 2           | 999        | 2024-01-01  | 4      | Good experience overall. |  |
| 24                   | 3           | 999        | 2024-01-01  | 2      | Late delivery.           |  |
| NULL                 | NULL        | NULL       | NULL        | NULL   | NULL                     |  |

## Step 31: Error Logging Verification

The parsing\_error\_log table was checked to verify whether any parsing or validation errors occurred during the CSV, JSON, and XML integration process. Since all source records passed validation successfully, no rejected records or parsing errors were found in the error log table.

```
410
411 -- Error logging verification
412
413 • SELECT *
414 FROM parsing_error_log;
415
```



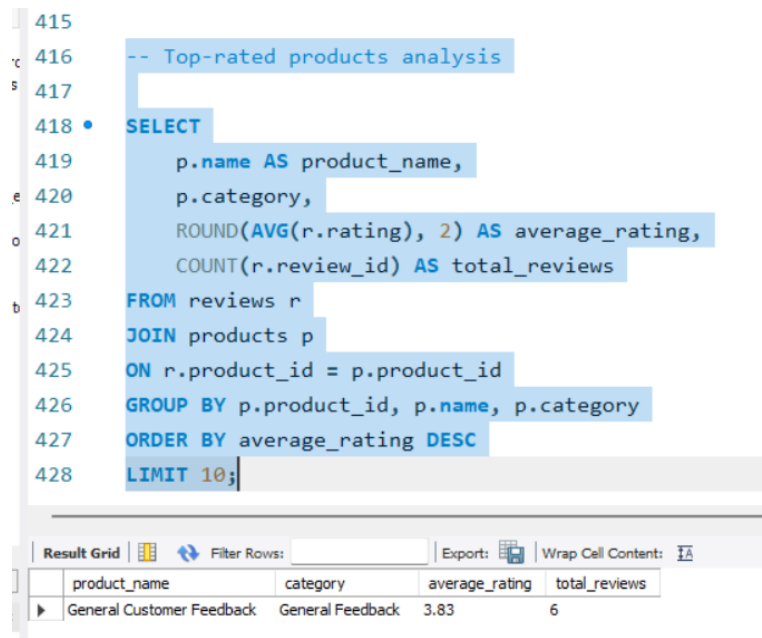
|   | log_id | source_file | row_identifier | error_description | log_timestamp |
|---|--------|-------------|----------------|-------------------|---------------|
| * | NULL   | NULL        | NULL           | NULL              | NULL          |

## Part 5: Analysis and Presentation

### Step 32: Top-Rated Products Analysis

A SQL analysis query was executed to calculate the average product ratings and total review counts using GROUP BY and aggregate functions. The results identified the top-performing product category based on customer review ratings.

```
415
416 -- Top-rated products analysis
417
418 • SELECT
419     p.name AS product_name,
420     p.category,
421     ROUND(AVG(r.rating), 2) AS average_rating,
422     COUNT(r.review_id) AS total_reviews
423 FROM reviews r
424 JOIN products p
425 ON r.product_id = p.product_id
426 GROUP BY p.product_id, p.name, p.category
427 ORDER BY average_rating DESC
428 LIMIT 10;
```



| product_name              | category         | average_rating | total_reviews |
|---------------------------|------------------|----------------|---------------|
| General Customer Feedback | General Feedback | 3.83           | 6             |

## Step 33: Common Complaints Analysis Using Keywords

A SQL query was used to identify recurring customer complaints by searching for keywords such as “late”, “damaged”, and “defective” within customer comments. A CASE statement was used to classify complaint types, and the results were grouped by product category to count complaint occurrences. The analysis identified “Late delivery” as one of the detected customer complaints.

```
438
439 -- Common complaints analysis by keyword and category
440
441 SELECT
442     p.category,
443
444     CASE
445         WHEN LOWER(r.comments) LIKE '%late%' THEN 'Late delivery'
446         WHEN LOWER(r.comments) LIKE '%damaged%' THEN 'Damaged product'
447         WHEN LOWER(r.comments) LIKE '%defective%' THEN 'Defective product'
448         ELSE 'Other'
449     END AS complaint_type,
450
451     COUNT(*) AS complaint_count
452
453 FROM reviews r
454 JOIN products p
455 ON r.product_id = p.product_id
456
457 WHERE LOWER(r.comments) LIKE '%late%'
458     OR LOWER(r.comments) LIKE '%damaged%'
459     OR LOWER(r.comments) LIKE '%defective%'
460
461 GROUP BY p.category, complaint_type
462
```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents:

|                     | category      | complaint_type | complaint_count |
|---------------------|---------------|----------------|-----------------|
| ▶ General Feedback: | Late delivery |                | 1               |

## Step 34: Customer Sentiment Classification Based on Ratings

A SQL CASE statement was used to classify customer reviews into Positive, Neutral, and Negative sentiment categories based on rating values. Reviews with ratings of 4 and 5 were classified as Positive, rating 3 as Neutral, and ratings below 3 as Negative. This analysis helped evaluate overall customer satisfaction levels.

```
454 -- Customer sentiment classification based on ratings
455
456 SELECT
457     review_id,
458     customer_id,
459     rating,
460     comments,
461
462     CASE
463         WHEN rating >= 4 THEN 'Positive'
464         WHEN rating = 3 THEN 'Neutral'
465         ELSE 'Negative'
466     END AS sentiment
467
468 FROM reviews;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents:

|   | review_id | customer_id | rating | comments                 | sentiment |
|---|-----------|-------------|--------|--------------------------|-----------|
| ▶ | 16        | 1           | 5      | Great product!           | Positive  |
|   | 17        | 2           | 4      | Fast delivery.           | Positive  |
|   | 18        | 3           | 3      | Average quality.         | Neutral   |
|   | 22        | 1           | 5      | Excellent service!       | Positive  |
|   | 23        | 2           | 4      | Good experience overall. | Positive  |
|   | 24        | 3           | 2      | Late delivery.           | Negative  |

## Step 35: Trends Over Time in Ratings and Reviews

A SQL analysis query was used to track trends in customer ratings and review activity over time. The data was grouped by review month to calculate average ratings and total review counts. The results showed that all reviews were submitted during January with an average rating of 3.83 across 6 total reviews.

```
477 -- Trends over time in ratings and reviews
478
479 • SELECT
480     MONTH(review_date) AS review_month,
481     ROUND(AVG(rating), 2) AS average_rating,
482     COUNT(review_id) AS total_reviews
483 FROM reviews
484 GROUP BY MONTH(review_date)
485 ORDER BY review_month;
```

| review_month | average_rating | total_reviews |
|--------------|----------------|---------------|
| 1            | 3.83           | 6             |

## Step 36: Product Categories with Highest Satisfaction Scores

A SQL aggregation query was executed to calculate customer satisfaction scores by product category. The query used average rating calculations and review counts to identify categories with the highest satisfaction levels. The results showed that the “General Feedback” category achieved an average satisfaction score of 3.83 based on 6 customer reviews.

```
488
489 -- Product categories with highest satisfaction scores
490
491 • SELECT
492     p.category,
493     ROUND(AVG(r.rating), 2) AS average_satisfaction_score,
494     COUNT(r.review_id) AS total_reviews
495 FROM reviews r
496 JOIN products p
497 ON r.product_id = p.product_id
498 GROUP BY p.category
499 ORDER BY average_satisfaction_score DESC;
```

| category         | average_satisfaction_score | total_reviews |
|------------------|----------------------------|---------------|
| General Feedback | 3.83                       | 6             |

## **Step 37 — Recommendations for Product and Service Improvement**

Based on the analysis results, several recommendations can be made to improve customer satisfaction and product performance. The complaint analysis identified “Late delivery” as a recurring issue, suggesting that the company should improve shipping and delivery processes. The sentiment analysis showed that most customer reviews were positive, but some negative and neutral reviews indicated opportunities for service improvement. Monitoring customer feedback regularly and responding quickly to customer concerns can help improve overall satisfaction levels. Additionally, tracking product ratings over time can support better business decisions and improve customer experience.

## **Conclusion**

This project demonstrated the complete ETL process using structured and semi-structured customer feedback data from CSV, JSON, and XML sources. Data was successfully parsed, validated, cleaned, transformed, and loaded into a MySQL database for analysis. SQL queries were used to identify top-rated products, detect customer complaints, classify customer sentiment, and analyze satisfaction trends over time. The project showed how ETL and data analysis techniques can support business decision-making and improve customer satisfaction through data-driven insights.

## Step 39: Presentation Slides Preparation

The presentation slides were prepared to summarize the main parts of the project, including the database design, ETL workflow, data parsing process, validation steps, SQL analysis results, key findings, and recommendations. The slides provide a clear overview of how CSV, JSON, and XML feedback data were integrated, cleaned, analyzed, and used to support business decision-making.

## Database Design / ER Diagram

The ER diagram from Final Project Part 1 was used as the main database design for this continuation project. The same database structure was used to maintain relationships between Customers, Products, and Reviews. Additional staging tables were used in Part 2 to temporarily store parsed CSV, JSON, and XML data before loading clean records into the final Reviews table.

